Wording for NB comment resolution on trivial relocation

Document #:	P3920R0 [Latest] [Status]
Date:	2025-11-07
Project:	Programming Language C++
Audience:	EWG, LWG, CWG

Reply-to:	Louis Dionne <ldionne.2@gmail.com>

Contents

§ 1 Summary

In a joint EWG and LEWG session in Kona, there was consensus to remove the trivial relocation feature (P2786R13) from C++26 due to various issues that won't be repeated here, with a desire to fix these issues in the C++29 time frame. This paper contains the wording for the removal, based on top of the current working draft.

§ 2 Effect on Trivial Relocation NB comments

§ 2.1 Resolve the following NB Comments as accepted

Revert P2786:

- ✅ US 45-081 11.1 `[class.pre]` Satisfy expectations that trivially relocatably types are memcpy-able
- ✅ US 9-024 5.11, 6.09.1, 7.5.6.2, 11.1, 11.2, 16.4.6.11, 21.3, 21.4, A.9, C.1.4, C.1.6 Excise "replaceable type"

Remove the `std::relocate` function:

-

— ✅ CA-133 20.2.6 [obj.lifetime] Remove std::relocate

— ✅ US 73-131 20.2.2, 20.2.6 Cleaning up the trivial relocation APIs

§ 2.2 Resolve the following NB comments as moot

Respecify using memcpy rules:

— ✅ BDS 2-034 6.8.1 [basic.types.general], 11.2 [class.prop], 20.2.6 [obj.lifetime] Permit trivial relocation via memcpy

— ✅ CN 6.9.1p9 [basic.types.general] Trivial relocatability should imply bitwise relocatability

— ✅ US 46-085 11.2, 20.2.6 Make trivial relocation implementable by memcpy

Fix the special tokens:

— ✅ US 44-082 11.1 [class.pre] Conditional trivially relocatable types

— ✅ FR-002-025 5.12 Table 5 [lex.key] Make *_if_eligible real keywords

— ✅ US 10-023 5.11, 11.01, 11.2, A.9, C.1.4, C.1.6 Remove *_if_eligible keywords

Support more types:

— ✅ CA-136 20.2.6p16 [obj.lifetime] Allow const-qualified types for relocate

— ✅ CA-137 20.2.6p9 [obj.lifetime] Allow const-qualified types for trivially_relocate

Fix implicit property:

— ✅ US 47-084 11.2p2 [class.prop] Implicitly deleted move operation should not disable trivial relocation CWG3049

§ 2.3 The following NB comments mentioning trivial relocation remain unresolved by this paper

Adopt uninitialized relocation algorithm:

— US 72-130 20 [mem] adopt std::uninitialized_relocate (P3516)

— US 166-266 26.11.1 [specialized.algorithms.general] P3516 Adopt Uninitialized algorithms for relocation (for C++26)

Adopt new lifetime management primitive:

— RO 1-134 20.2.6 [obj.lifetime] Add a start_lifetime_at function P3858

— US 74-135 20.2.6 [obj.lifetime] Add a start_lifetime_at function

§ 3 Wording

This wording does a partial revert of what was added by P2786R13. Some drive-by improvements made in P2786R13 are conserved, and this paper calls them out.

This paper also removes a few additions that were made with Reflection (P2996R13) in Sofia.

§ 3.1 Undo changes in 5.11 [lex.name]

Table 4: Identifiers with special meaning [tab:lex.name.special]:

final	import	post	~~replaceable_if_eligible~~
override	module	pre	~~trivially_relocatable_if_eligible~~

§ 3.2 Undo changes in 6.9.1 [basic.types.general]

9 Arithmetic types ([basic.fundamental]), enumeration types, pointer types, pointer-to-member types ([basic.compound]), std::meta::info, std::nullptr_t, and cv-qualified versions of these types are collectively called *scalar types*. Scalar types, trivially copyable class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivially copyable types*. ~~Scalar types, trivially relocatable class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivially relocatable types*. Cv-unqualified scalar types, replaceable class types ([class.prop]), and arrays of such types are collectively called *replaceable types*.~~ Scalar types, standard-layout class types ([class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *standard-layout types*. Scalar types, implicit-lifetime class types ([class.prop]), array types, and cv-qualified versions of these types are collectively called *implicit-lifetime types*.

§ 3.3 Undo changes in 7.5.6.2 [expr.prim.lambda.closure]

4 The closure type is not an aggregate type (9.5.2 [dcl.init.aggr]); it is a structural type (13.2 [temp.param]) if and only if the lambda has no lambda-capture. An implementation may define the closure type differently from what is described below provided this does not al-

ter the observable behavior of the program other than by changing

(4.1) — the size and/or alignment of the closure type,

(4.2) — whether the closure type is trivially copyable (11.2 [class.prop]), or

(4.3) ~~— whether the closure type is trivially relocatable (11.2 [class.prop]),~~

(4.4) ~~— whether the closure type is replaceable (11.2 [class.prop]), or~~

(4.x) — whether the closure type is a standard-layout class (11.2 [class.prop]).

§ 3.4 Partially undo changes in 11.1 [class.pre]

When applying the wording for P2786R13, improvements were made to how the class specifiers are defined. CWG expressed a desire to keep those improvements, so this is not a pure revert of the changes added by P2786R13.

1 […]

class-head:
class-key attribute-specifier-seq$_{opt}$ *class-head-name class-property-specifier-seq*$_{opt}$ *base-clause*$_{opt}$
class-key attribute-specifier-seq$_{opt}$ *base-clause*$_{opt}$

class-head-name:
nested-name-specifier$_{opt}$ *class-name*

class-property-specifier-seq:
class-property-specifier class-property-specifier-seq$_{opt}$

class-property-specifier:
`final`
~~`trivially_relocatable_if_eligible`~~
~~`replaceable_if_eligible`~~

[…]

5 Each *class-property-specifier* shall appear at most once within a single *class-property-specifier-seq*. Whenever a *class-key* is followed by a *class-head-name*, ~~one of the identifiers `final`, `trivially_relocatable_if_eligible`, or `replaceable_if_eligible`~~the identifier `final`, and a colon or left brace, the identifier is interpreted as a *class-property-specifier*.

[*Example:*

```
struct A;
struct A final {};      // OK, definition of struct A,
                        // not value-initialization of variable final

struct X {
 struct C { constexpr operator int() { return 5; } };
 struct B trivially_relocatable_if_eligible final : C{};
                        // OK, definition of nested class B,
                        // not declaration of a bit-field member
                        // trivially_relocatable_if_eligible final
};
```

— *end example*]

§ 3.5 Undo changes in 11.2 [class.prop]

2 A class C is *default-movable* if

(2.1) — overload resolution for direct-initializing an object of type `C` from an xvalue of type `C` selects a constructor that is a direct member of `C` and is neither user-provided nor deleted,

(2.2) — overload resolution for assigning to an lvalue of type `C` from an xvalue of type `C` selects an assignment operator function that is a direct member of `C and is neither user-provided nor deleted, and

(2.3) — `C` has a destructor that is neither user-provided nor deleted.

3 A class is *eligible for trivial relocation* unless it

(3.1) — has any virtual base classes,

(3.2) — has a base class that is not a trivially relocatable class,

(3.3) — has a non-static data member of an object type that is not of a trivially relocatable type, or

(3.4) — has a deleted destructor, except that it is implementation-defined whether an otherwise-eligible union having one or more subobjects of polymorphic class type is eligible for trivial relocation.

4 A class `C` is a *trivially relocatable class* if it is eligible for trivial relocation and

(4.1) — has the `trivially_relocatable_if_eligible` *class-property-specifier*,

(4.2) — is a union with no user-declared special member functions, or

(4.3) — is default-movable.

5 [*Note:* A class with const-qualified or reference non-static data members can be trivially relocatable. — *end note*]

6 A class `C` is *eligible for replacement* unless

(6.1) — it has a base class that is not a replaceable class,

(6.2) — it has a non-static data member that is not of a replaceable type,

(6.3) — overload resolution fails or selects a deleted constructor when direct-initializing an object of type `C` from an xvalue of type `C` ([dcl.init.general]),

(6.4) — overload resolution fails or selects a deleted assignment operator function when assigning to an lvalue of type `C` from an xvalue of type `C` ([expr.assign], [over.assign]), or

(6.5) — it has a deleted destructor.

7 A class `C` is a *replaceable class* if it is eligible for replacement and

(7.1) — has the `replaceable_if_eligible` *class-property-specifier*,

(7.2) — is a union with no user-declared special member functions, or

(7.3) — is default-movable.

8 [*Note:* Accessibility of the special member functions is not considered when establishing trivial relocatability or replaceability. — *end note*]

9 [*Note:* Not all trivially copyable classes are trivially relocatable or replaceable. — *end note*]

§ 3.6 Undo changes in 15.12 [cpp.predefined]

Table 22 — Feature-test macros [tab:cpp.predefined.ft]

Macro name	Value
…	…
`__cpp_threadsafe_static_init`	`200806L`
~~`__cpp_trivial_relocatability`~~	~~`202502L`~~
`__cpp_trivial_union`	`202502L`
…	…

§ 3.7 Undo changes in C.1.4 [diff.cpp23.dcl.dcl]

1 **Affected subclause:** [dcl.decl.general]
Change: Introduction of `trivially_relocatable_if_eligible` and `replaceable_if_eligible` as identifiers with special meaning ([lex.name]).
Rationale: Support declaration of trivially relocatable and replaceable types ([class.prop]).
Effect on original feature: Valid C++ 2023 code can become ill-formed.

[*Example:*

```
struct C {};
struct C replaceable_if_eligible {};    // was well-formed (new variable replaceable_if_eligible)
                                        // now ill-formed (redefines C)
```

— *end example*]

§ 3.8 Note on changes in C.6.4 [diff.cpp03.dcl.dcl]

P2786R13 also did a drive-by fix to add an entry for `final` in [diff.cpp03.dcl.dcl]. We are not undoing that drive-by fix in this paper.

§ 3.9 Partially undo changes in 16.4.6 [conforming]

16.4.6.11 [library.class.props] **Properties of library classes**

1 Unless explicitly stated otherwise, it is unspecified whether any class described in [support] through [exec] and [depr] is a trivially copyable class, a standard-layout class, or an implicit-lifetime class ([class.prop]).

2 ~~Unless explicitly stated otherwise, it is unspecified whether any class for which trivial relocation (i.e., the effects of `trivially_relocate` ([obj.lifetime])) would be semantically equivalent to move-construction of the destination object followed by destruction of the source object is a trivially relocatable class ([class.prop]).~~

3 ~~Unless explicitly stated otherwise, it is unspecified whether a class `C` is a replaceable class ([class.prop]) if assigning an xvalue `a` of type `C` to an object `b` of type `C` is semantically equivalent to destroying `b` and then constructing from `a` in `b`'s place.~~

Note that paragraph 1 was also added by P2786R13, but we are not removing it since it appears to be an intentional drive-by change.

§ 3.10 Undo changes in 21.3.3 [meta.type.synop]

```
// [meta.unary.prop], type properties
template<class T> struct is_const;
template<class T> struct is_volatile;
template<class T> struct is_trivially_copyable;
template<class T> struct is_trivially_relocatable;
template<class T> struct is_replaceable;
template<class T> struct is_standard_layout;
template<class T> struct is_empty;
template<class T> struct is_polymorphic;
template<class T> struct is_abstract;
template<class T> struct is_final;
template<class T> struct is_aggregate;
template<class T> struct is_consteval_only;
...
template<class T> struct is_nothrow_destructible;
template<class T> struct is_nothrow_relocatable;

...

// [meta.unary.prop], type properties
template<class T>
  constexpr bool is_const_v = is_const<T>::value;
template<class T>
  constexpr bool is_volatile_v = is_volatile<T>::value;
template<class T>
  constexpr bool is_trivially_copyable_v = is_trivially_copyable<T>::value;
template<class T>
  constexpr bool is_trivially_relocatable_v = is_trivially_relocatable<T>::value;
template<class T>
  constexpr bool is_standard_layout_v = is_standard_layout<T>::value;
...
template<class T>
  constexpr bool is_implicit_lifetime_v = is_implicit_lifetime<T>::value;
template<class T>
  constexpr bool is_replaceable_v = is_replaceable<T>::value;
template<class T>
  constexpr bool has_virtual_destructor_v = has_virtual_destructor<T>::value;
...
template<class T>
  constexpr bool is_nothrow_destructible_v = is_nothrow_destructible<T>::value;
template<class T>
  constexpr bool is_nothrow_relocatable_v = is_nothrow_relocatable<T>::value;
template<class T>
  constexpr bool is_implicit_lifetime_v = is_implicit_lifetime<T>::value;
...
```

§ 3.11 Undo changes in 21.3.6.4 [meta.unary.prop]

Table 54 — Type property predicates [tab:meta.unary.prop]

Template	Condition	Preconditions
…		
`template<class T>` `struct is_trivially_relocatable;`	T is a trivially relocatable type ([basic.types.general])	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.
`template<class T>` `struct is_replaceable;`	T is a replaceable type ([basic.types.general])	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.

Template	Condition	Preconditions

`template<class T>`
`struct is_nothrow_relocatable;`
	`is_trivially_relocatable_v<T> ||`
	`(is_nothrow_move_constructible_v<remove_all_extents_t<T>>`
	`&& is_nothrow_destructible_v<remove_all_extents_t<T>>)`
		`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.

…		

§ 3.12 Undo changes in 20.2.2 [memory.syn]

```
// [obj.lifetime], explicit lifetime management
template<class T>
  T* start_lifetime_as(void* p) noexcept;                                    // freestanding
template<class T>
  const T* start_lifetime_as(const void* p) noexcept;                        // freestanding
template<class T>
  volatile T* start_lifetime_as(volatile void* p) noexcept;                  // freestanding
template<class T>
  const volatile T* start_lifetime_as(const volatile void* p) noexcept;      // freestanding
template<class T>
  T* start_lifetime_as_array(void* p, size_t n) noexcept;                    // freestanding
template<class T>
  const T* start_lifetime_as_array(const void* p, size_t n) noexcept;        // freestanding
template<class T>
  volatile T* start_lifetime_as_array(volatile void* p, size_t n) noexcept;  // freestanding
template<class T>
  const volatile T* start_lifetime_as_array(const volatile void* p,          // freestanding
                                            size_t n) noexcept;
~~template<class T>~~
  ~~T* trivially_relocate(T* first, T* last, T* result);~~                   ~~// freestanding~~
~~template<class T>~~
  ~~constexpr T* relocate(T* first, T* last, T* result);~~                   ~~// freestanding~~
```

§ 3.13 Undo changes in 20.2.6 [obj.lifetime]

```
template<class T>
  T* trivially_relocate(T* first, T* last, T* result);
```

9 *Mandates*: `is_trivially_relocatable_v<T> && !is_const_v<T>` is `true`. `T` is not an array of unknown bound.

10 *Preconditions*:

(10.1) — `[first, last)` is a valid range.

(10.2) — `[result, result + (last - first))` denotes a region of storage that is a subset of the region reachable through result ([basic.compound]) and suitably aligned for the type `T`.

(10.3) — No element in the range `[first, last)` is a potentially-overlapping subobject.

11 *Postconditions*: No effect if `result == first` is `true`. Otherwise, the range denoted by `[result, result + (last - first))` contains objects (including subobjects) whose lifetime has begun and whose object representations are the original object representations of the corresponding objects in the source range `[first, last)` except for any parts of the object representations used by the implementation to represent type information ([intro.object]). If any of the objects has union type, its active member is the same as that of the corresponding object in the source range. If any of the aforementioned objects has a non-static data member of reference type, that reference refers to the same entity as does the corresponding reference in the source range. The lifetimes of the original objects in the source range have ended.

12 *Returns*: `result + (last - first)`.

13 *Throws*: Nothing.

14 *Complexity*: Linear in the length of the source range.

15 *Remarks*: The destination region of storage is considered reused ([basic.life]). No constructors or destructors are invoked.

[*Note:* Overlapping ranges are supported. — *end note*]

```
template<class T>
  constexpr T* relocate(T* first, T* last, T* result);
```

16 *Mandates*: `is_nothrow_relocatable_v<T> && !is_const_v<T>` is `true`. `T` is not an array of unknown bound.

17 *Preconditions*:

(17.1) — `[first, last)` is a valid range.

(17.2) — `[result, result + (last - first))` denotes a region of storage that is a subset of the region reachable through result ([basic.compound]) and suitably aligned for the type `T`.

(17.3) — No element in the range `[first, last)` is a potentially-overlapping subobject.

18 *Effects*:

(18.1) — If `result == first` is `true`, no effect;

(18.2) — otherwise, if not called during constant evaluation and `is_trivially_relocatable_v<T>` is `true`, then has effects equivalent to: `trivially_relocate(first, last, result);`

(18.3) — otherwise, for each integer `i` in `[0, last - first)`,

(18.3.1) — if `T` is an array type, equivalent to: `relocate(begin(first[i]), end(first[i]), *start_lifetime_as<T>(result + i));`

(18.3.2) — otherwise, equivalent to: `construct_at(result + i, std::move(first[i])); destroy_at(first + i);`

19 *Returns*: `result + (last - first)`.

20 *Throws*: Nothing.

[*Note:* Overlapping ranges are supported. — *end note*]

§ 3.14 Undo changes in C.1.6 [diff.cpp23.library]

2 **Affected subclause**: [res.on.macro.definitions]
Change: Additional restrictions on macro names.
Rationale: Avoid hard to diagnose or non-portable constructs.
Effect on original feature: Names of special identifiers may not be used as macro names. Valid C++ 2023 code that defines `replaceable_if_eligible` or `trivially_relocatable_if_eligible` as macros is invalid in this revision of C++.

§ 3.15 Undo changes in 17.3.2 [version.syn]

2 Each of the macros defined in `<version>` is also defined after inclusion of any member of the set of library headers indicated in the corresponding comment in this synopsis.

[*Note:* Future revisions of this document might replace the values of these macros with greater values. — *end note*]

```
...
#define __cpp_lib_transformation_trait_aliases      201304L // freestanding, also in <type_traits>
#define __cpp_lib_transparent_operators             201510L // freestanding, also in <memory>, <functional>
#define __cpp_lib_trivially_relocatable             202502L // freestanding, also in <memory>, <type_traits>
#define __cpp_lib_tuple_element_t                   201402L // freestanding, also in <tuple>
#define __cpp_lib_tuple_like                                  202311L // also in <utility>, <tuple>, <map>,
       <unordered_map>
...
```

§ 3.16 Remove 21.4.1 [meta.syn] changes added by P2996R13 in Sofia

```
// associated with [meta.unary.prop], type properties
consteval bool is_const_type(info type);
consteval bool is_volatile_type(info type);
consteval bool is_trivially_copyable_type(info type);
consteval bool is_trivially_relocatable_type(info type);
consteval bool is_replaceable_type(info type);
consteval bool is_standard_layout_type(info type);
consteval bool is_empty_type(info type);

...

consteval bool is_nothrow_destructible_type(info type);
consteval bool is_nothrow_relocatable_type(info type);
```

§ 3.17 Remove 21.4.17 [meta.reflection.traits] changes added by P2996R13 in Sofia

```
// associated with [meta.unary.prop], type properties
consteval bool is_const_type(info type);
consteval bool is_volatile_type(info type);
consteval bool is_trivially_copyable_type(info type);
consteval bool is_trivially_relocatable_type(info type);
consteval bool is_replaceable_type(info type);
consteval bool is_standard_layout_type(info type);
consteval bool is_empty_type(info type);

...

consteval bool is_nothrow_destructible_type(info type);
consteval bool is_nothrow_relocatable_type(info type);
```

§ 4 Acknowledgements

Thanks to Alisdair Meredith for reviewing the wording in this paper and for providing the list of NB comment dispositions.